MODULE 04 · BLOCK 1 - COMPUTER PROGRAMMING AND PYTHON FUNDAMENTALS

Comparison, Boolean and bitwise operators

How Python compares values and strings, what and and or really return, bitwise tricks, and the full precedence table.

BLOCK 1 of 4 EXAM WEIGHT 18% MODULE 04 of 28

admin@sthithapragna.com

Comparisons always give True or False. **and** and **or** do not: they hand back one of their operands, and stop the moment the answer is known.

Comparisons give a bool

OPERATOR	MEANING	EXAMPLE	RESULT
==	equal to	3 == 3.0	True
!=	not equal to	3 != 3	False
< >	less than, greater than	2 < 10	True
<= >=	less or equal, greater or equal	5 >= 5	True

THE TRAP

= assigns and == compares. There is no <> in Python 3, and =< is a SyntaxError: it must be <=.

Numbers mix. Strings and numbers do not

```
1  print(1 == 1.0)      # True   int and float compare by value
2  print(True == 1)     # True   True is 1 underneath
3  print("1" == 1)      # False  a str never equals an int
4  print(None == 0)     # False
5  print("1" < 1)       # TypeError: '<' not supported
```

1-2 Numbers compare by value, whatever their type. bool counts as a number.

3 Equality between a str and an int is allowed, and always False.

5 Ordering a str against an int is not allowed at all. It raises TypeError.

1 < x < 10 means 1 < x and x < 10

```
1  x = 5
2  print(1 < x < 10)      # True   1 < 5 and 5 < 10
3  print(10 > x > 7)      # False  10 > 5 and 5 > 7
4  print(x == 5 > 4)      # True   x == 5 and 5 > 4
5  print(1 < 3 > 2)       # True   legal, if odd
```

2 A chain is an and of neighbouring pairs. The middle value is shared.

4 == chains too. It does not mean (x == 5) > 4.

Strings compare character by character

```
1  print("apple" < "banana")   # True   a comes before b
2  print("Mike" < "Mikey")     # True   a prefix is smaller
3  print("Zebra" < "apple")    # True   capitals come first
4  print("10" < "9")           # True   '1' comes before '9'
5  print(ord("A"), ord("a"), ord("0"))   # 65 97 48
```

1-2 Compare the first differing character. If one string runs out first, it is smaller.

3-4 The order is by code point, not the alphabet. Digits, then capitals, then lower case.

ASCII, Unicode and code points

- **ASCII** defines 128 characters, codes 0 to 127. Enough for English, not for the world.
- **Unicode** gives every character in every script a number. Its first 128 match ASCII.
- That number is the **code point**. ord() gives it, and chr() turns it back into a character.
- **UTF-8** is an encoding: a way to store code points as bytes. Python source is UTF-8 by default.
- **I18N** is short for internationalisation: 18 letters between the I and the N.

Every value is either truthy or falsy

FALSY: TREATED AS FALSE	TRUTHY: EVERYTHING ELSE, INCLUDING...
`False None`	True
`0 0.0`	-1 0.001
`"" the empty string`	" " "0" "False"
`[] () {} empty collections`	[0] [False] (None,)

THE TRAP

Only empty and zero values are falsy. A string containing a space, a zero or the word False is truthy.

and and or return an operand, not always a bool

```
1  print(not 0)          # True   not always returns a bool
2  print(3 and 5)        # 5      both truthy: the last one
3  print(0 and 5)        # 0      first falsy value
4  print(3 or 5)         # 3      first truthy value
5  print(0 or "x")       # x
6  print("" or 0)        # 0      all falsy: the last one
```

2-3 and returns the first falsy operand, or the last one if all are truthy.

4-6 or returns the first truthy operand, or the last one if all are falsy.

Once the answer is certain, Python stops

```
1  print(0 and 1 / 0)     # 0      and already failed: 1 / 0 never runs
2  print(1 or 1 / 0)      # 1      or already succeeded
3  print(1 and 1 / 0)     # ZeroDivisionError: must check the right side
```

1-2 The right side is skipped entirely, so its error never happens.

3 Here the left side decides nothing, so the right side runs and fails.

Operator precedence, highest first

OPERATORS	NOTES
`**`	right to left
`+x -x ~x`	unary; all three share a level
`* / // %`	
`+ -`	binary
`<< >>`	shifts
`& then ^ then |`	three separate levels, in this order
`== != < <= > >= in is`	all comparisons share one level
`not`	
`and`	
`or`	lowest

Four expressions the table decides

```
1  print(not 1 == 2)              # True   not (1 == 2)
2  print(True or False and False) # True   True or (False and False)
3  print(2 & 3 == 2)              # True   (2 & 3) == 2
4  print(1 + 2 << 1)              # 6      (1 + 2) << 1
```

1. not is below ==, so the comparison happens first.
2. and binds tighter than or. Evaluate the and, then the or.

Where study material gets the table wrong

- Some banks group **not** with unary plus and minus, near the top. That is wrong.
- The unary operator beside minus is **~**, bitwise not. Logical **not** sits below every comparison.
- Test: not 1 == 2 is True. If not bound tighter, it would be (not 1) == 2, which is False.
- If every option makes this mistake, choose by the order of the other levels.

> REMEMBER IT
>
> Brackets, power, unary, multiply, add, shifts, bitwise, comparisons, not, and, or. Say it once a day.

Operating on individual bits

OPERATOR	NAME	A BIT IN THE RESULT IS 1 WHEN...
&	bitwise and	both bits are 1
|	bitwise or	either bit is 1
^	bitwise xor	the bits differ
~	bitwise not	always: ~x is -x - 1
<< >>	shift left, shift right	x << n is x * 2**n; x >> n is x // 2**n

ON THE EXAM

Bitwise operators need integers. ~1.5 is a TypeError.

6 and 3, bit by bit

```
1  # 6 is 0b110 and 3 is 0b011
2  print(6 & 3)     # 2    0b010  both bits 1
3  print(6 | 3)     # 7    0b111  either bit 1
4  print(6 ^ 3)     # 5    0b101  bits differ
5  print(~6)        # -7   -6 - 1
6  print(6 << 2)    # 24   6 * 4
7  print(6 >> 1)    # 3    6 // 2
```

2-4 Write both numbers in binary, one above the other, and compare column by column.

5 Flip and step back: the answer is the negative, minus one more.

and is not &, and or is not |

LOGICAL

Truthiness, and short-circuits

```
print(1 and 2)   # 2
print(1 or 2)    # 1
print(0 and 1/0) # 0
```

Returns an operand. Skips the right side when it can.

BITWISE

Bits, and always both sides

```
print(1 & 2)     # 0
print(1 | 2)     # 3
print(0 & 1/0)   # ZeroDivisionError
```

Works column by column. Always evaluates both sides.

ON THE EXAM

On True and False, & and | give the same answers as and and or. On other numbers, they usually do not.

== asks the same value. is asks the same object

```
1  a = [1, 2]
2  b = [1, 2]
3  c = a
4  print(a == b, a is b, a is c)    # True False True
```

1-2 Two separate lists that happen to contain the same values.

3 No copy is made. c is a second sticky note on the same list as a.

Use == to compare values. Use is only to test for None: if x is None.

What to carry into the exam

- **and** returns the first falsy operand, **or** the first truthy one. Both short-circuit.
- Falsy: False, None, 0, 0.0, empty strings and empty collections. " " is truthy.
- Strings compare by code point: digits, capitals, then lower case.
- Order: comparisons, then not, then and, then or. **not** is not a unary arithmetic operator.
- ~x is -x - 1. Shifts multiply or floor divide by powers of two.

What is the output of this program?

```
1  a = 0
2  b = "yes"
3  print(a or b, a and b)
```

A True False

B yes 0

C yes False

D 1 0

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  a = 0
2  b = "yes"
3  print(a or b, a and b)
```

A `True False`

Assumes and and or return bools. They return an operand.

B `yes 0`

or skips the falsy 0 and returns "yes"; and stops at 0 and returns it.

C `yes False`

Right for or, but and returns the operand 0, not False.

D `1 0`

Converts "yes" to 1. Nothing converts it.

THE SIGNAL

If a question uses and or or outside an if, expect an operand back, not True or False.

What is the output of this program?

```
1  x, y, z = True, False, False
2  print(x or y and z, (x or y) and z)
```

A True True

B False False

C True False

D False True

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  x, y, z = True, False, False
2  print(x or y and z, (x or y) and z)
```

x or (y and z) is True or False, so True. (x or y) and z is True and False, so False.

A `True True`
Ignores the brackets in the second expression.

B `False False`
Reads the first expression left to right, like the second.

C `True False`
and binds first in the first; the brackets force or first in the second.

D `False True`
Swaps the two results.

THE SIGNAL

and binds tighter than or. Without brackets, group the and first.

What is the output of this program?

```
1  x = 5
2  print(1 < x < 10, 10 > x > 7, x == 5 > 4)
```

A `True False False`

B `True True True`

C The code raises a SyntaxError

D `True False True`

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  x = 5
2  print(1 < x < 10, 10 > x > 7, x == 5 > 4)
```

x == 5 > 4 means x == 5 and 5 > 4: True and True.

A `True False False`

Reads the third as (x == 5) > 4, that is True > 4. It is a chain.

B `True True True`

5 > 7 is False, so the second chain fails.

C The code raises a SyntaxError

Chaining is legal with any comparison operators, == included.

D `True False True`

Each chain is an and of neighbouring pairs.

THE SIGNAL

a op1 b op2 c always means a op1 b and b op2 c, whatever the operators.

> Each expression is evaluated on its own.

Which two expressions evaluate to True? (Choose two.)

A `'Mike' > 'Mikey'`

B `'mike' > 'Mike'`

C `'10' < '9'`

D `'Zoo' > 'apple'`

◆ PAUSE AND DECIDE

Which two expressions evaluate to True? (Choose two.)

A `'Mike' > 'Mikey'`

Mike runs out first, so it is the smaller string.

B `'mike' > 'Mike'`

m is 109, M is 77. Lower case comes after capitals.

C `'10' < '9'`

Only the first characters matter: '1' is 49, '9' is 57.

D `'Zoo' > 'apple'`

Z is 90, a is 97. Every capital comes before every lower case letter.

THE SIGNAL

Compare code points, not the alphabet: digits, then capitals, then lower case.

What is the output of this program?

```
1  a = 6
2  b = 3
3  print(a & b, a | b, a ^ b)
```

A. False True True

B. 2 7 7

C. 3 7 5

D. 2 7 5

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  a = 6
2  b = 3
3  print(a & b, a | b, a ^ b)
```

110 & 011 = 010 (2). 110 | 011 = 111 (7). 110 ^ 011 = 101 (5).

A `False True True`

Treats the bitwise operators as logical ones. They return ints.

B `2 7 7`

Treats ^ as or. Exclusive or drops the column where both are 1.

C `3 7 5`

Takes the smaller number for &. It compares bits, not sizes.

D `2 7 5`

Column by column: both 1, either 1, bits differ.

THE SIGNAL

Write both numbers in binary, one above the other, and work column by column.

What is the output of this program?

```
1  x = 5
2  print(~x, x << 2, x >> 1)
```

A -6 20 2

B -5 20 2

C -6 10 2.5

D 2 20 2

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  x = 5
2  print(~x, x << 2, x >> 1)
```

~5 is -5 - 1. 5 << 2 is 5 * 4. 5 >> 1 is 5 // 2.

A `-6 20 2`

Flip and step back gives -6. Left 2 is times 4; right 1 floor-halves.

B `-5 20 2`

Treats ~ as a plain minus. It is minus, then one more.

C `-6 10 2.5`

Shifts by one place only, and uses / not //. Shifts stay ints.

D `2 20 2`

Flips only three bits, 101 to 010. Python ints have no fixed width.

THE SIGNAL

~x is -x - 1. x << n is x * 2**n. x >> n is x // 2**n.

What is the output of this program?

```
1 print(bool(0.0), bool(" "), bool([0]), bool("False"))
```

A False False False False

B False True False False

C False True True True

D True True True True

◆ PAUSE AND DECIDE

What is the output of this program?

```
1 print(bool(0.0), bool(" "), bool([0]), bool("False"))
```

A `False False False False`

Judges each by what it looks like. Only 0.0 is zero or empty.

B `False True False False`

Looks inside the list and the string. bool only asks if they are empty.

C `False True True True`

0.0 is zero; the rest are non-empty, so truthy.

D `True True True True`

0.0 is a zero, and every zero is falsy.

THE SIGNAL

For strings and collections, ask one question: is it empty? For numbers: is it zero?

What is the output of this program?

```
1  a = [1, 2]
2  b = [1, 2]
3  c = a
4  print(a == b, a is b, a is c)
```

A True False True

B True True True

C False False True

D True False False

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  a = [1, 2]
2  b = [1, 2]
3  c = a
4  print(a == b, a is b, a is c)
```

A `True False True`

Equal values, different objects; c is a second name for a's list.

B `True True True`

Treats is like ==. Two list literals make two lists.

C `False False True`

a and b hold the same values, so == is True.

D `True False False`

Assumes c = a copies the list. Assignment never copies.

THE SIGNAL

== compares values. is compares identity. Assignment adds a name; it never copies.